Name-Vineet Gajwani

Div T1,Roll no 2401028

Branch – AI&DS

Experiment-04

AIM: To understand Continuous integration, Jenkins installation.

LO mapped: LO3

Theory:

Continuous Integration is a development practice that promotes collaboration, early issue detection, and automated testing to enhance the quality and efficiency of software development.

Jenkins is an open-source automation server that facilitates continuous integration and continuous delivery (CI/CD) of software projects. It helps developers automate various stages of the software development lifecycle, from building and testing to deployment and monitoring. Jenkins supportsa wide range of plugins and integrations, making it a versatile tool for automating various tasks in software development.

Main Functions:

- Automates build, testing, and deployment.
- Supports thousands of plugins for Git, Maven, Docker, Kubernetes, AWS, etc.
- Can trigger jobs automatically through Git webhooks, schedules (cron), or manually.

Jenkins Architecture

1. Jenkins Controller (Master)

- Manages jobs and configurations.
- Schedules builds and assigns them to agents.
- Provides the Jenkins Web UI.

2. Jenkins Agent (Node)

- Executes build, test, and deployment tasks.
- Allows distributed builds across multiple machines.

Types of Jenkins Pipelines

A Jenkins Pipeline automates the complete CI/CD workflow using a **Jenkinsfile**.

A. Declarative Pipeline

- Written inside the pipeline {} block.
- Simple, structured, and easy to read.
- Provides built-in syntax validation.
- Uses sections like agent, stages, stage, and steps.
- Best for standard CI/CD workflows.

B. Scripted Pipeline

- Written inside the node {} block.
- Uses Groovy scripting language.
- Highly flexible and customizable.
- Suitable for complex workflows with conditions and loops.
- More difficult to learn than Declarative Pipeline.

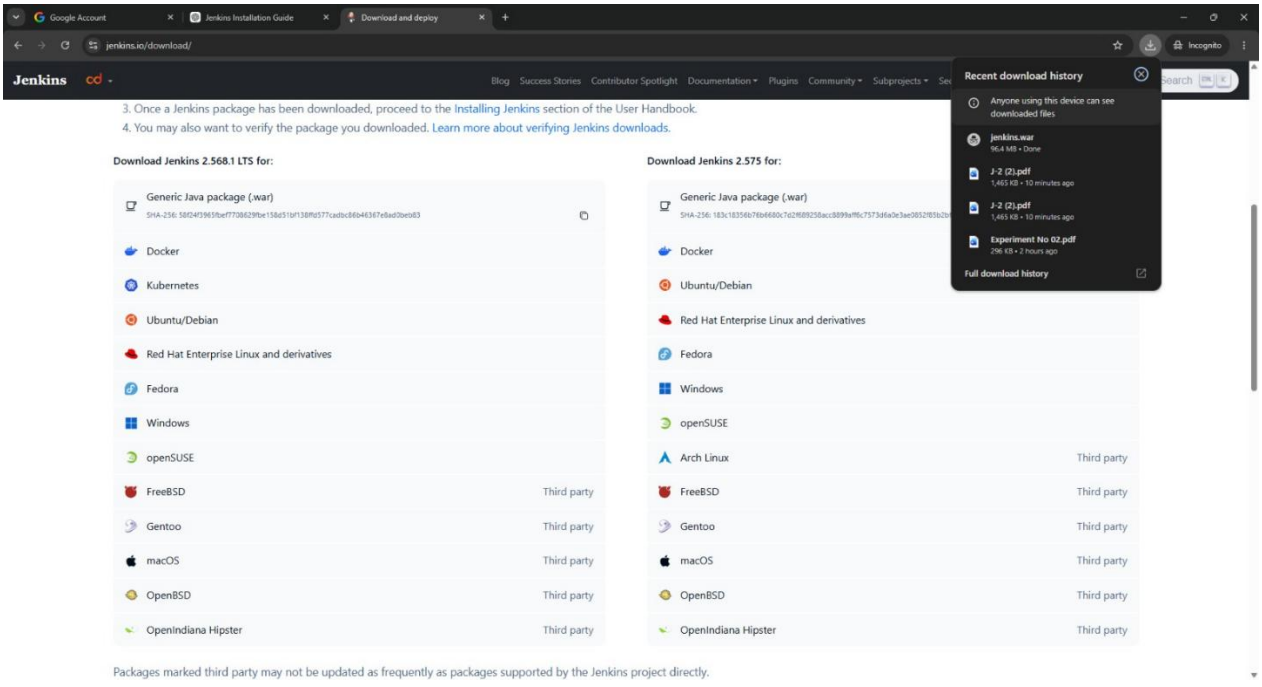
OUTPUT:

Step 1: Java (JDK) Installation on Verification

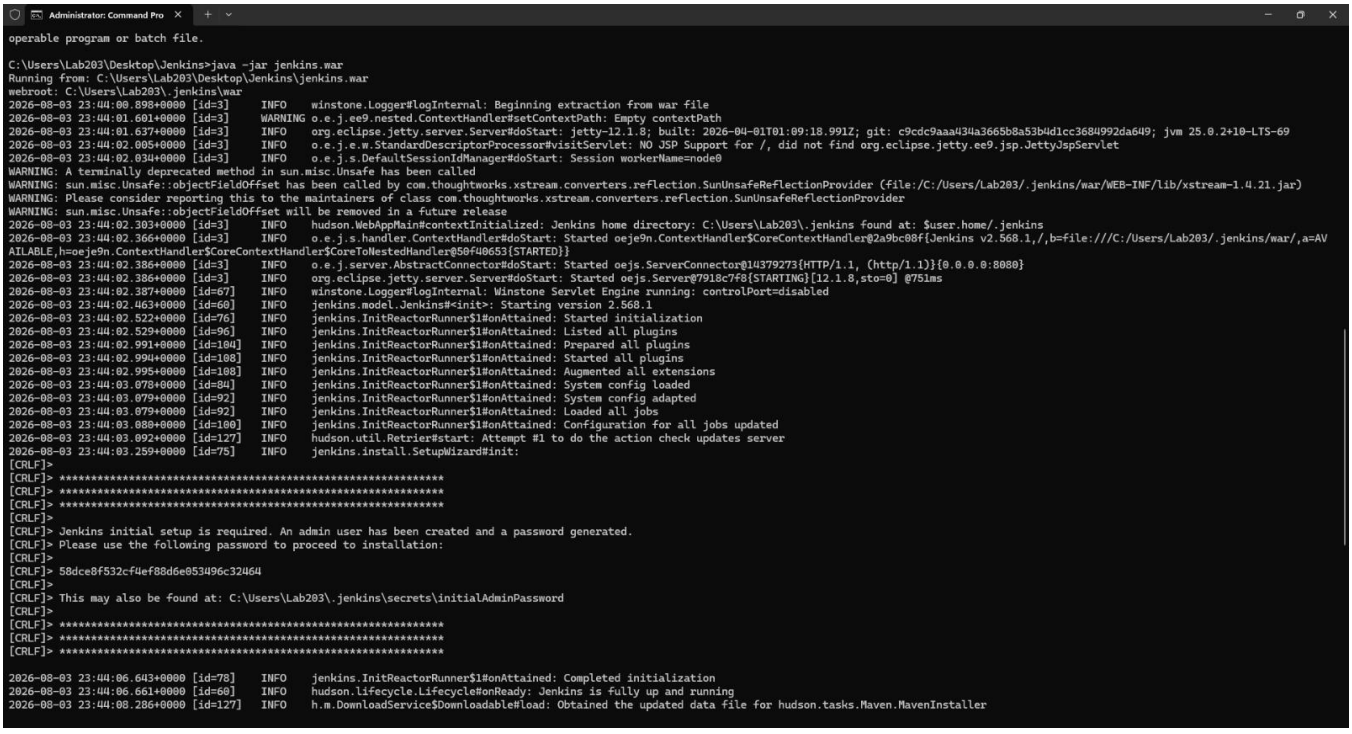
```
C:\Users\Lab203>java --version
java 25.0.2 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed mode, sharing)
```

Step 2: Jenkins Server Launch

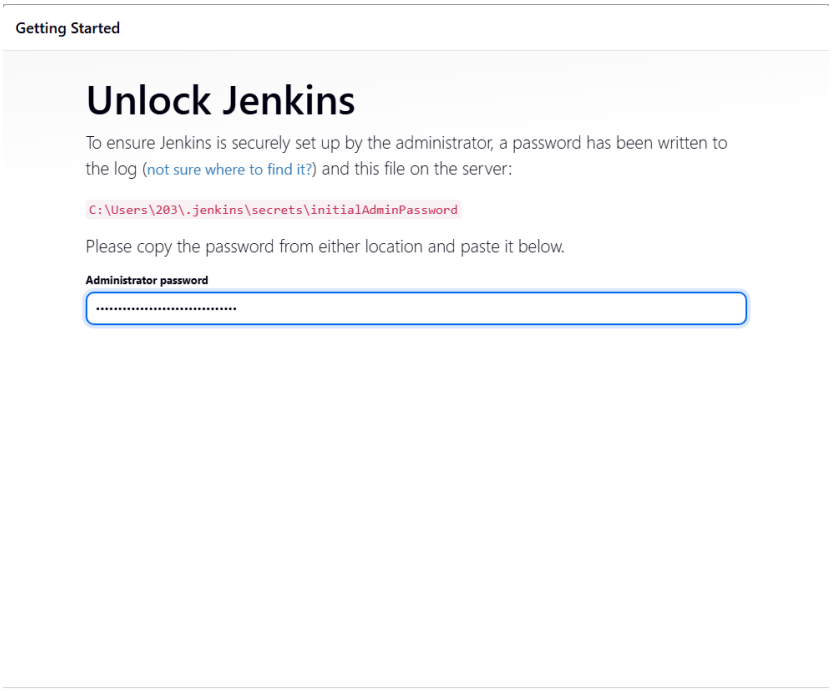
Downloaded the Generic Java package (jenkins.war) from the official website.



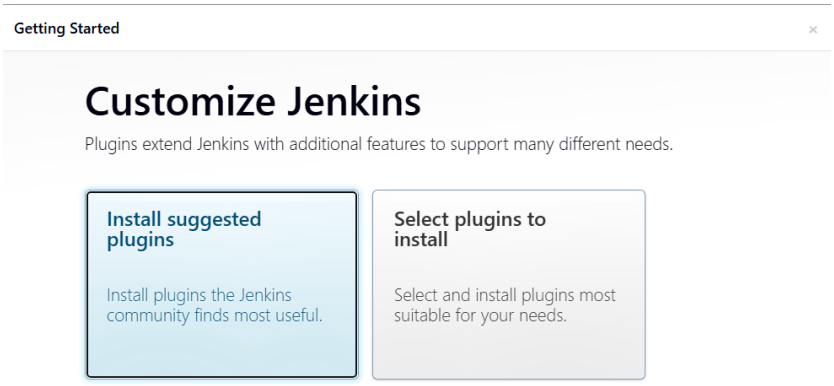
Executed Jenkins using the --enable-future-java flag to support JDK 22:

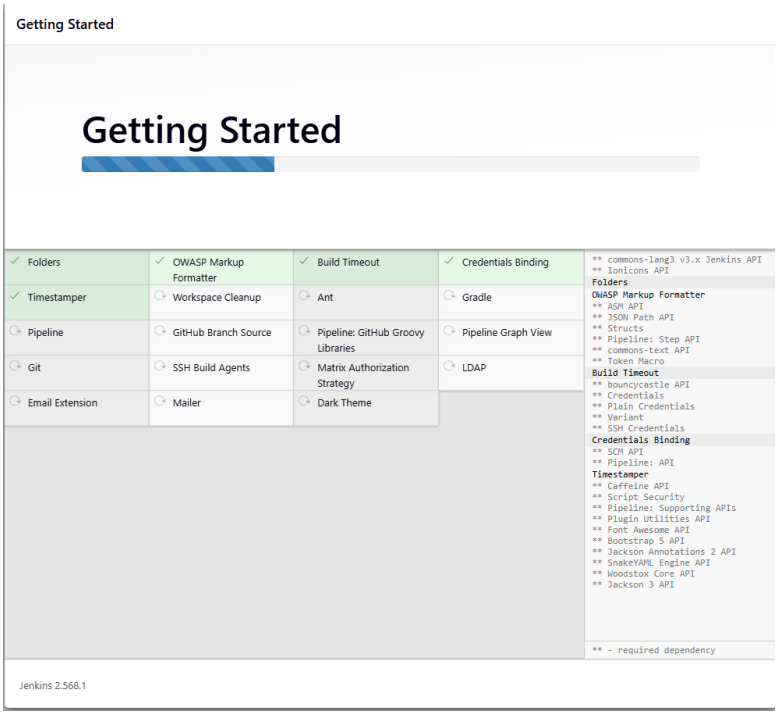


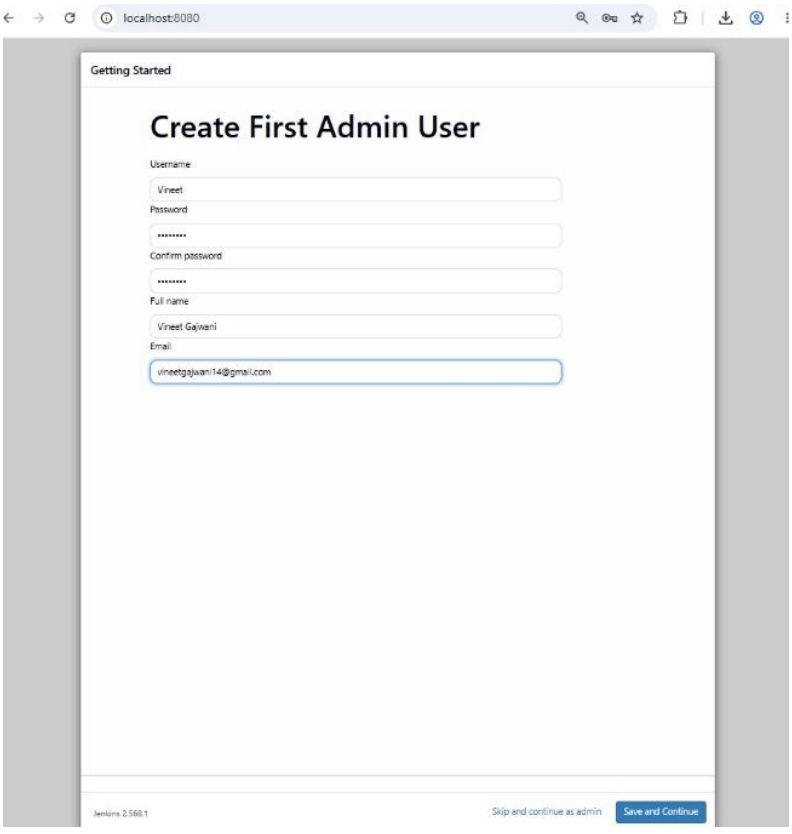
Step 3: Web Setup & Admin Configuration

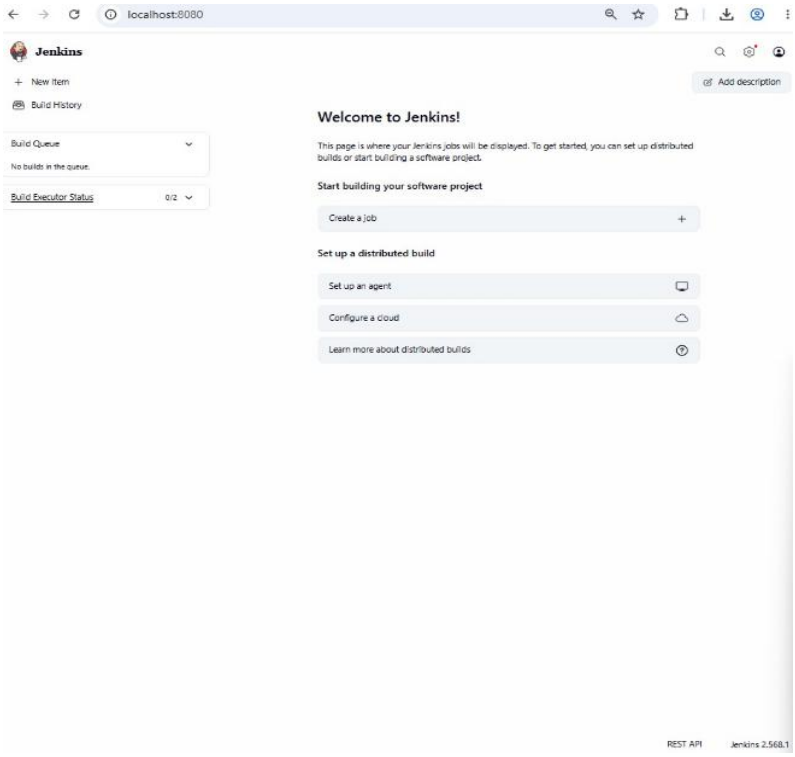




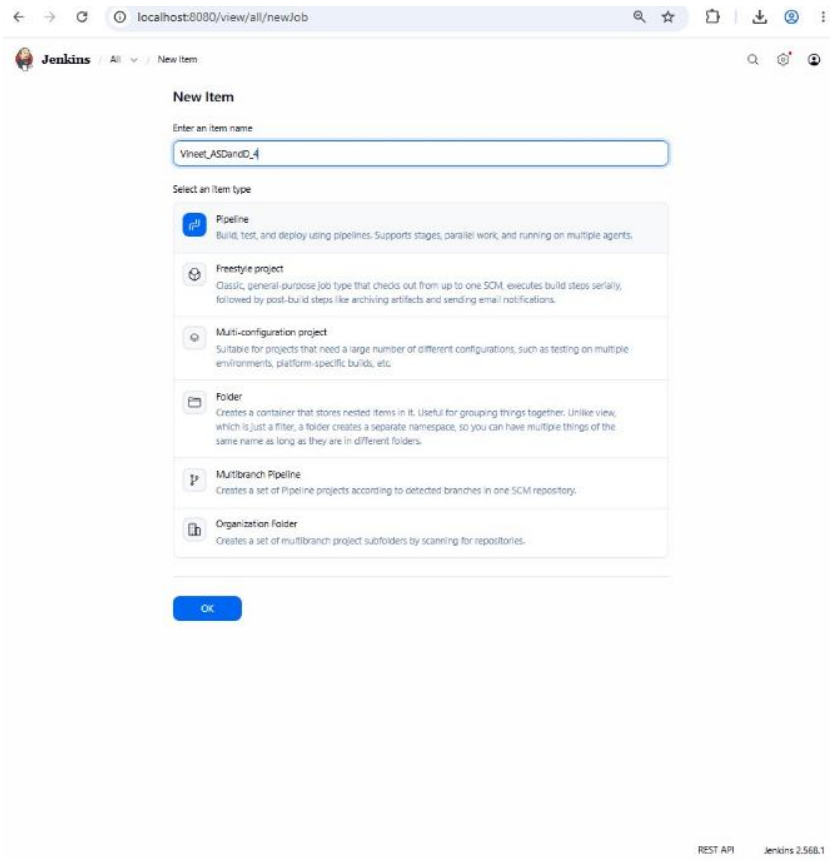



Getting Started
Getting Started
Folders
OWASP Markup Formatter
Build Timeout
Credentials Binding
Timestamper
Workspace Cleanup
Ant
Gradle
Pipeline
GitHub Branch Source
Pipeline: GitHub Groovy Libraries
Pipeline Graph View
Git
SSH Build Agents
Matrix Authorization Strategy
LDAP
Email Extension
Mailer
Dark Theme
** commons-lang3 v3.x Jenkins API
** Ionicons API
Folders
OWASP Markup Formatter
** ASM API
** JSON Path API
** Structs
** Pipeline: Step API
** commons-text API
** Token Macro
Build Timeout
** bouncycastle API
** Credentials
** Plain Credentials
** Variant
** SSH Credentials
Credentials Binding
** SCM API
** Pipeline: API
Timestamper
** Caffeine API
** Script Security
** Pipeline: Supporting APIs
** Plugin Utilities API
** Font Awesome API
** Bootstrap 5 API
** Jackson Annotations 2 API
** SnakeYAML Engine API
** Woodstox Core API
** Jackson 3 API
** - required dependency
Jenkins 2.568.1


localhost:8080
Getting Started
Create First Admin User
Username
Vineet
Password
Confirm password
Full name
Vineet Gajwani
Email
vineetgajwani14@gmail.com
Jenkins 2.568.1
Skip and continue as admin
Save and Continue

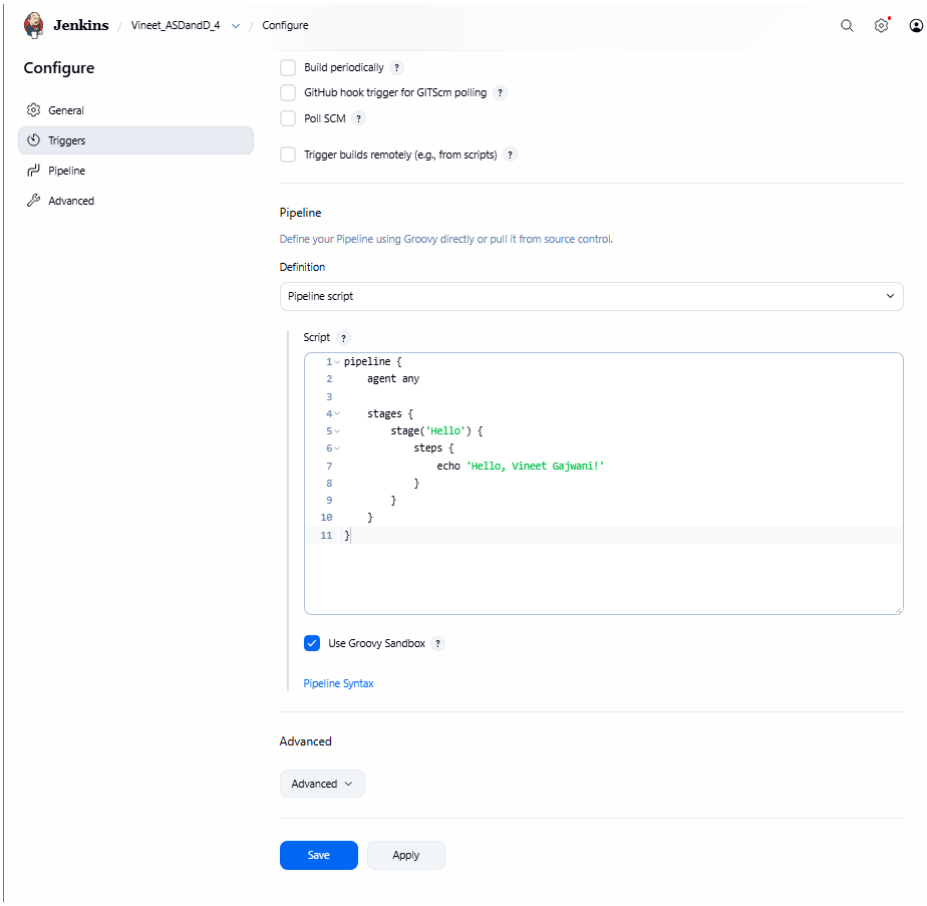


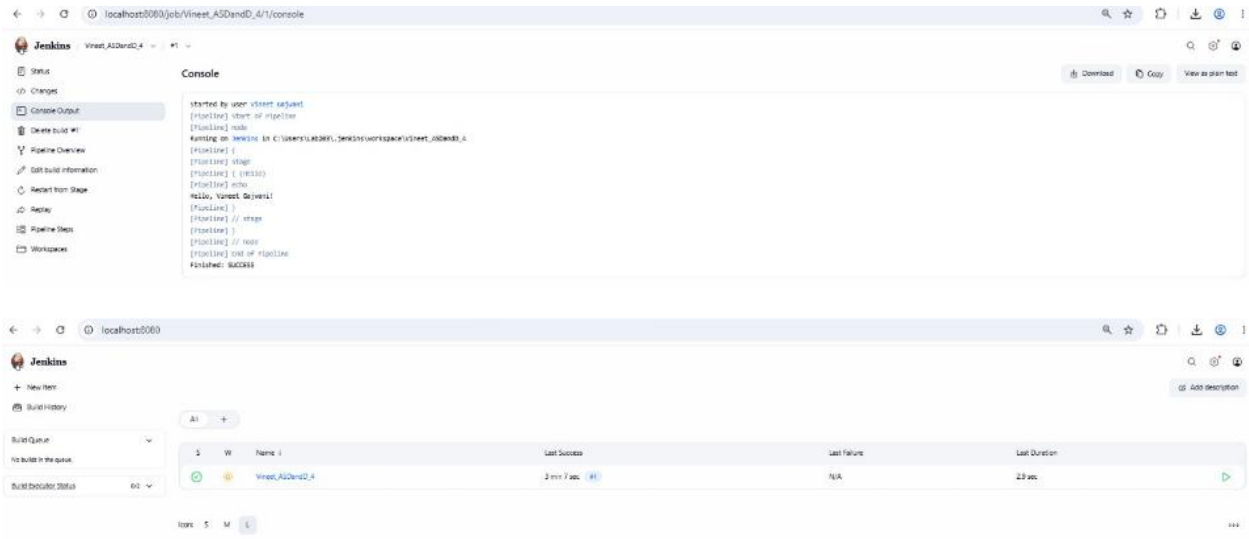
Step 4: Creating Pipelines



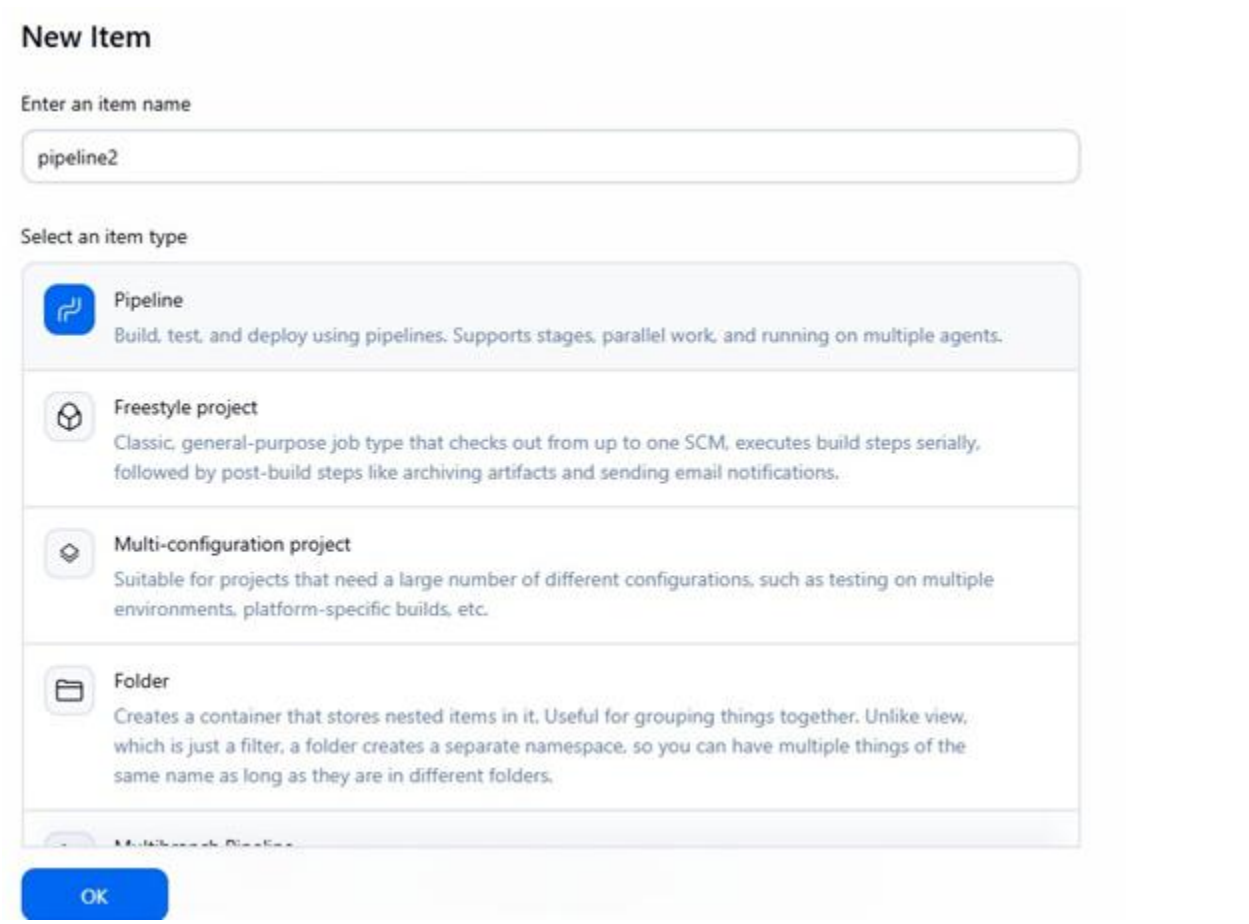
Code:

```
pipeline {
        agent any
        stages {
                stage('Hello') {
                        steps {
                                echo 'Hello, Vineet Gajwani!'
                        }
                }
        }
}
```





Step 5: Creating Scripted Pipeline



Code:

```
node {
    stage('Hello') {
        echo 'Hello, Scripted Jenkins Pipeline!'
    }
}
```

Console

Download Copy Vie

```
Started by user Vineet Gajwani
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\Users\Lenovo\.jenkins\workspace\Pipeline2
[Pipeline] {
[Pipeline] stage (hide)
[Pipeline] { (Hello)
[Pipeline] echo
Hello, Jenkins!!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

CONCLUSION: LO1, LO3

In this experiment, we learned how to create Jenkins Pipelines to automate the build, test, and deployment process. We explored both Declarative and Scripted pipelines, understanding their structure, benefits, and use cases. By implementing different pipeline examples, we demonstrated how Jenkins enables efficient, repeatable, and reliable CI/CD workflows.